TECH BYTES

Kubernetes in an hour

Architecture, networking and the workloads you will actually run

60 minutes, for engineers who already build and ship applications

AGENDA

The plan for the hour

Minutes	Segment	You leave with
0-2	Welcome	The plan and the handout
2-7	Why, and the core idea	Desired state and reconciliation
7-17	Architecture	Control plane, workers, failure modes
17-24	Networking	Node, Pod and Service ranges
24-40	Core objects	Pod to Deployment, Service, config
40-48	Workload types	Deployment, StatefulSet, Job
48-57	Live demo	Deploy, break, update, roll back
57-60	Wrap up	Takeaways and next topics

WHY KUBERNETES

What running containers at scale demands

Scheduling

Which machine has room for this container?

Self-healing

Replace what crashes, without a pager.

Scaling

Add or remove copies as load changes.

Rollouts

Ship a new version with no downtime.

Discovery

Find a service even as its IPs change.

Configuration

Inject settings and secrets per environment.

Kubernetes answers all six with one model: declare what you want.

THE CORE IDEA

Declare the state, let controllers converge

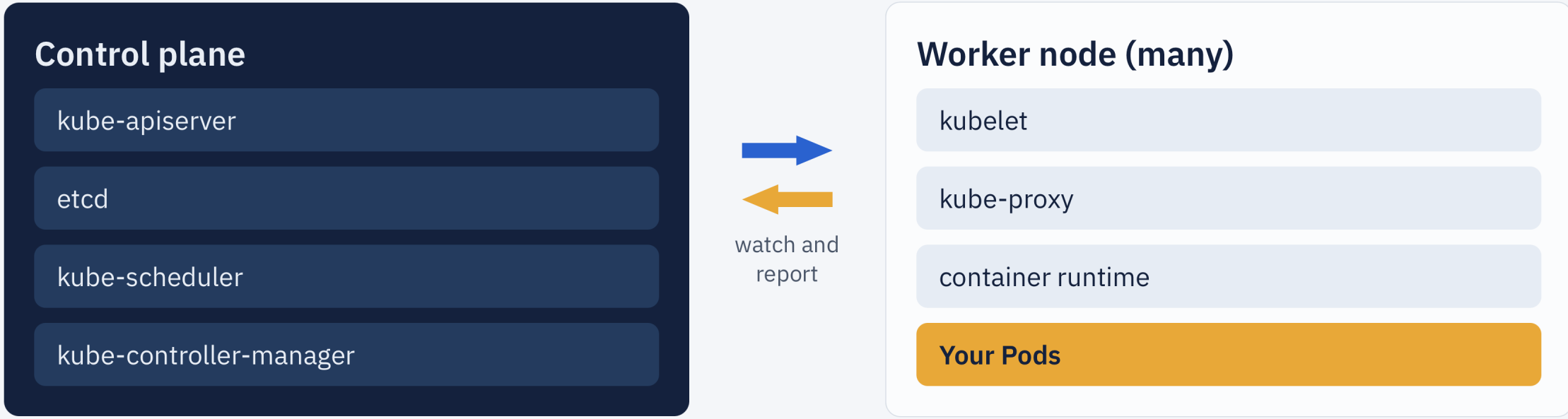
1. You declare: 3 replicas of web, image nginx 1.27
2. The API server stores it in etcd
3. A controller compares desired with actual
4. It creates, deletes or restarts until they match

```
while true:
  actual = observe()
  if actual ≠ desired:
    act(actual, desired)
```

Every Kubernetes controller is this loop: Deployment, Job, Node, Service.

ARCHITECTURE

Control plane and worker nodes



Components coordinate through the API server rather than calling each other.

ARCHITECTURE

What each control plane component does

Component	Job	Think of it as
kube-apiserver	Validates and stores every object	The API gateway
etcd	Consistent store of all cluster state	The database
kube-scheduler	Assigns each new Pod to a node	A constraint solver
kube-controller-manager	Runs the reconcile loops	Many small watchers
cloud-controller-manager	Bridges to cloud load balancers	The cloud adapter

> If it decides, it is control plane. If it runs your container, it is a worker.

ARCHITECTURE

Life of a kubectl apply

1. kubectl sends the Deployment to the API server, which stores it in etcd
2. The Deployment controller creates a ReplicaSet; that controller creates Pods
3. The scheduler sees unassigned Pods and picks a node for each
4. The kubelet on that node tells the container runtime to start them
5. The CNI plugin and kube-proxy wire up Pod and Service networking
6. The kubelet reports status back; the controllers keep watching

Nothing calls anything directly: each step reacts to a change in the API.

ARCHITECTURE

What breaks, and what Kubernetes does

Failure	What happens	What you see
Container crashes	kubelet restarts it with backoff	Restart count rises
Pod is deleted	ReplicaSet creates a replacement	New Pod with a new IP
Worker node dies	Node goes NotReady; Pods evicted later	Pods move after ~5 min by default
Control plane down	Running Pods keep serving traffic	No new changes; apps stay up

> Design for it: more than one replica, readiness probes, spread across nodes.

NETWORKING

Three address ranges, kept apart

Range	Who gets an IP	Example	Set by
Node subnet	The machines themselves	10.0.0.0/24	Your network or VPC
Pod CIDR	Every Pod	10.244.0.0/16	CNI plugin
Service CIDR	Every Service ClusterIP	10.96.0.0/12	API server flag

> Plan once: the ranges must not overlap each other or your corporate network, and they are painful to change later.

NETWORKING

How the ranges fit together

Node subnet 10.0.0.0/24: real, routable addresses

node-1 · 10.0.0.11

Pod slice 10.244.1.0/24

`10.244.1.5` `10.244.1.6`

node-2 · 10.0.0.12

Pod slice 10.244.2.0/24

`10.244.2.8` `10.244.2.9`

node-3 · 10.0.0.13

Pod slice 10.244.3.0/24

`10.244.3.4`

Service CIDR 10.96.0.0/12: virtual

web-svc at 10.96.45.20 is a rule on every node that forwards to a healthy Pod IP above.

NETWORKING

The rules of the Kubernetes network

Flat Pod network

Every Pod can reach every other Pod by IP, with no NAT.

Services

A stable virtual IP in front of a changing set of Pods.

Cluster DNS

Each Service gets a DNS name, resolved by CoreDNS.

```
web-svc.default.svc.cluster.local → 10.96.45.20
```

Pod IPs are ephemeral. Talk to a Service, never to a Pod IP.

CORE OBJECTS

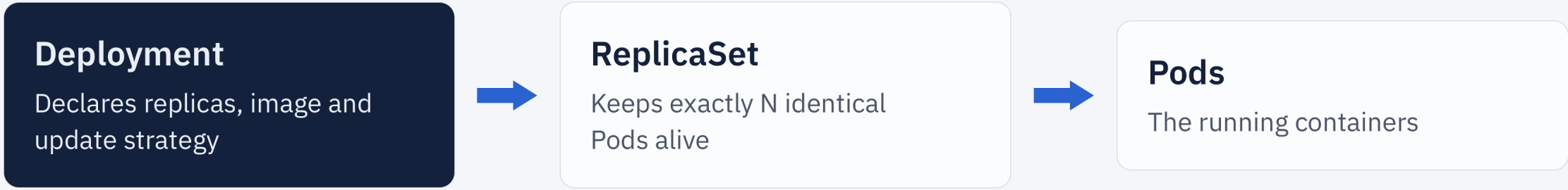
Pod: the smallest deployable unit

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    app: hello
spec:
  containers:
  - name: web
    image: nginx:1.27
    ports:
    - containerPort: 80
```

- One or more containers sharing one IP and storage
- Ephemeral: a replacement gets a new IP
- You rarely create bare Pods; controllers do
- Labels are how everything else finds them

CORE OBJECTS

Deployment owns ReplicaSet owns Pods



- You edit the Deployment; each change creates a new ReplicaSet
- Old ReplicaSets are kept at zero replicas, so rollback is instant

CORE OBJECTS

A Deployment manifest

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: nginx:1.27
```

- The selector must match the template labels
- readinessProbe gates traffic; livenessProbe restarts
- requests and limits drive scheduling
- Change the image and a rolling update starts

CORE OBJECTS

Rolling update, step by step

Step	What happens	Old Pods	New Pods
1	Start	3	0
2	Create 1 new Pod	3	1 starting
3	New Pod ready, stop 1 old	2	1
4	Repeat	1	2
5	Finish	0	3

With maxUnavailable 0 and a readiness probe, capacity never drops below 3.

```
kubectl rollout undo deployment/web
```

CORE OBJECTS

Service: a stable address for Pods

Type	Reachable from
ClusterIP	Inside the cluster only (default)
NodePort	Any node IP on ports 30000-32767
LoadBalancer	A cloud load balancer in front
Headless	Returns Pod IPs directly; for StatefulSets

```
kind: Service
metadata: {name: web-svc}
spec:
  type: ClusterIP
  selector: {app: web}
  ports:
  - port: 80
    targetPort: 80
```

The selector, not the Pod name, decides which Pods receive traffic.

CORE OBJECTS

ConfigMap and Secret

ConfigMap

```
kind: ConfigMap
metadata: {name: app-config}
data:
  LOG_LEVEL: "info"
```

Non-secret settings as env vars or files.

Secret

```
kind: Secret
metadata: {name: app-secret}
stringData:
  DB_PASSWORD: change-me
```

Same shape, separate access control.

Twelve-factor in practice: one image everywhere, config injected per environment.

Base64 is encoding, not encryption

- Anyone who can read a Secret can decode it
- Turn on encryption at rest for etcd
- Restrict access with RBAC, or use an external secrets manager

WORKLOAD TYPES

Which workload type?

You need	Use	Pods are
Stateless service, rolling updates	Deployment	Interchangeable
Stable identity and per-Pod storage	StatefulSet	Named db-0, db-1, db-2
Run once to completion	Job	Retried until done
Run on a schedule	CronJob	A new Job each tick
One Pod on every node	DaemonSet	Agents, log shippers

Default to a Deployment; reach for the others when the workload demands it.

WORKLOAD TYPES

Job and CronJob

Job

```
kind: Job
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.40
```

Runs Pods to completion and retries failures. restartPolicy must be Never or OnFailure.

CronJob

```
kind: CronJob
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      # same as the Job spec
```

Creates a Job on a cron schedule. Forbid stops runs from overlapping.

WORKLOAD TYPES

StatefulSet: identity and storage that stick

db-0

Volume: data-db-0

`db-0.db-headless`

db-1

Volume: data-db-1

`db-1.db-headless`

db-2

Volume: data-db-2

`db-2.db-headless`

- Stable names and DNS through a headless Service
- Ordered start and stop
- Each replica keeps its own volume, even when rescheduled

LIVE DEMO

Demo 1: deploy and expose

```
kind create cluster
kubectl apply -f configmap.yaml -f deployment.yaml -f service.yaml
kubectl get pods -o wide
kubectl get svc
kubectl port-forward svc/web-svc 8080:80
```

- Pod IPs come from the Pod CIDR
- The ClusterIP comes from the Service CIDR
- Node IPs come from the node subnet

LIVE DEMO

Demo 2: break it, update it, roll back

```
kubectl delete pod -l app=web --wait=false
kubectl set image deployment/web web=nginx:1.27.1
kubectl rollout status deployment/web
kubectl rollout undo deployment/web
kubectl get events --sort-by=.lastTimestamp
```

- Delete Pods: the ReplicaSet replaces them with new IPs
- Update the image: watch the surge and the old ReplicaSet drop to zero
- Undo: the old ReplicaSet scales back up

WRAP UP

Three things to remember

1. You declare state; controllers converge reality to it.
2. Three address ranges: nodes, Pods and Services. Keep them separate.
3. Deployment for stateless, StatefulSet for identity, Job for finite work.

NEXT STEPS

Where to go from here

- Ingress and Gateway API
- PersistentVolumes and StorageClasses
- RBAC and NetworkPolicies
- Autoscaling and resource quotas
- Helm and GitOps

Take-home handout

Every manifest from today, ready to apply: Pod, ReplicaSet, Deployment, Service, ConfigMap, Secret, Job, CronJob and StatefulSet.

Questions?